Operational Guidance — pqc_crypto_module v0.1.0

> **Disclaimer**: This module is prepared for FIPS 140-3 evaluation and is not currently validated.

---

1. Prerequisites

- **Rust toolchain**: Nightly (required by the parent workspace). The module itself uses stable Rust features.
- **Operating system**: Linux (x86_64, aarch64) or macOS (aarch64) with a functioning OS-backed CSPRNG.
- **Dependencies**: All dependencies are fetched via Cargo. See `Cargo.toml` for the complete list.

2. Building the Module

Standard build (with legacy algorithm support)

```
cargo build -p pqc_crypto_module
```

Approved-only build (legacy algorithms excluded)

```
cargo build -p pqc_crypto_module --features approved-only
```

This eliminates all non-approved algorithms (Ed25519, SHA-256, HMAC-SHA256) from the compiled binary. Any code path that references `pqc_crypto_module::legacy::*` will fail to compile.

Running tests

```
# Unit tests
cargo test -p pqc_crypto_module --lib

# Integration tests
cargo test -p pqc_crypto_module --tests
```

```
# All tests
cargo test -p pqc_crypto_module
```

3. Module Initialization

The module must be initialized exactly once at process startup, before any cryptographic operation:

```
use pqc_crypto_module::api;

fn main() {
    // REQUIRED: Initialize approved mode (runs self-tests)
    api::initialize_approved_mode()
        .expect("cryptographic self-tests failed — cannot start");

    // Module is now in Approved state
    // All approved API functions are available
}
```

Initialization behavior

1. The module state transitions from `Uninitialized` to `SelfTesting`.
2. Four self-tests run sequentially: SHA3-256 KAT, ML-DSA-65 KAT, ML-KEM KAT, continuous RNG test.
3. If all tests pass, the state transitions to `Approved`.
4. If any test fails, the state transitions to `Error` and the function returns an error.

What happens if initialization fails

- All subsequent cryptographic operations return `Err(CryptoError::ModuleInErrorState)`.
- The module cannot recover. The process must be restarted.
- The application should log the error and exit or refuse to serve requests.

4. Using Approved Services

After successful initialization, approved services are available through the `api` module:

Digital signatures (ML-DSA-65)

```
use pqc_crypto_module::api;

// Generate keypair
let keypair = api::generate_mldsa_keypair()?;

// Sign a message
let signature = api::sign_message(&keypair.private_key,
        b"message")?;
```

```rust
// Verify a signature
api::verify_signature(&keypair.public_key, b"message",
        &signature)?;
```

Hashing (SHA3-256)

```rust
let hash = api::sha3_256(b"data to hash")?;
println!("digest: {}", hash.to_hex());
```

Key encapsulation (ML-KEM-768)

```rust
// Generate keypair
let kem_kp = api::generate_mlkem_keypair()?;

// Encapsulate (sender side)
let (ciphertext, shared_secret) =
        api::mlkem_encapsulate(&kem_kp.public_key)?;

// Decapsulate (receiver side)
let recovered_secret =
        api::mlkem_decapsulate(&kem_kp.private_key, &ciphertext)?;
```

Random byte generation

```rust
let random_data = api::random_bytes(32)?;
```

5. Error Handling

All API functions return `Result<T, CryptoError>`. The caller must handle errors explicitly:

Error	Meaning	Action
`ModuleNotInitialized`	`initialize_approved_mode()` was not called	Call `initialize_approved_mode()` first
`ModuleInErrorState`	Self-tests failed	Restart the process
`SelfTestFailed(desc)`	Specific self-test failure	Log the description, restart the process
`InvalidKey(desc)`	Key is wrong size or format	Check key source and encoding
`InvalidSignature`	Signature is wrong size or format	Check signature source and encoding
`VerificationFailed`	Signature did not verify	Signature is invalid for the given message/key
`RngFailure(desc)`	OS CSPRNG failed	Check OS entropy source, restart
`NonApprovedAlgorithm`	Legacy algorithm called in Approved mode	Use approved API instead

6. Checking Module State

The current module state can be queried at any time:

```
use pqc_crypto_module::approved_mode;

let state = approved_mode::state();
match state {
    approved_mode::ModuleState::Uninitialized => { /* not yet
        initialized */ }
    approved_mode::ModuleState::SelfTesting   => { /* tests
        running */ }
    approved_mode::ModuleState::Approved       =>
        { /* operational */ }
    approved_mode::ModuleState::Error          => { /* failed,
        restart needed */ }
}
```

7. Key Lifecycle Management

Generation

Keys are generated inside the module. The caller receives owned key types.

Storage

The module does not persist keys. The application layer is responsible for key storage. When storing keys:

- Use encrypted storage with access controls.
- Serialize via `as_bytes()` and deserialize via `from_bytes()`.
- Protect serialized key material at rest.

Destruction

Private keys (`MldsaPrivateKey`, `MlKemPrivateKey`) and shared secrets (`MlKemSharedSecret`) are automatically zeroized when they go out of scope. No explicit destruction call is needed.

To force immediate destruction:

```
drop(private_key); // Triggers ZeroizeOnDrop
```

8. Legacy Algorithm Usage

Legacy algorithms are available only before `initialize_approved_mode()` is called or when the module is not in `Approved` state.

```rust
use pqc_crypto_module::legacy;

// Only works BEFORE initialize_approved_mode()
let hash = legacy::legacy_sha256(b"old data")?;
```

After initialization, all legacy calls return `Err(CryptoError::NonApprovedAlgorithm)`.

To eliminate legacy algorithms entirely, build with `--features approved-only`.

9. Concurrency

The module is safe for concurrent use from multiple threads:

- The state machine uses `AtomicU8` with `SeqCst` ordering.
- All API functions are stateless (no shared mutable state beyond the module state).
- Key generation and signing can be called concurrently from different threads.
- Initialize the module once from a single thread before spawning worker threads.

10. Monitoring and Diagnostics

Startup verification

The application should verify initialization succeeded and log the result:

```rust
match api::initialize_approved_mode() {
    Ok(()) => log::info!("pqc_crypto_module: approved mode
        active"),
    Err(e) => {
        log::error!("pqc_crypto_module: self-test failure: {e}");
        std::process::exit(1);
    }
}
```

Runtime health check

Query `approved_mode::state()` periodically or in health-check endpoints to confirm the module remains in `Approved` state.

11. Environment Variables

The module itself does not read environment variables. The parent application uses:

Variable	Effect on module
`SIGNING_ALGORITHM`	Selects `ed25519` or `ml-dsa-65` at the application layer. When set to `ml-`

Variable	Effect on module
	`dsa-65`, all signing uses the approved ML-DSA path.

12. Security Considerations

- Call `initialize_approved_mode()` as early as possible in the process lifecycle.
- Do not suppress or ignore self-test failures.
- Do not use `__test_reset()` in production. It exists only for integration tests.
- Ensure the operating system provides adequate entropy (check `/proc/sys/kernel/random/entropy_avail` on Linux).
- When using `--features approved-only`, verify the build succeeds to confirm no legacy code paths remain.